

Documento de Modelo de Dominio y Decisiones de Diseño

Proyecto: Sistema de Gestión para Distribuidora de Bebidas

Tipo de proyecto: Trabajo Práctico Final – Tecnicatura en Programación

Integrantes: Enzo Leiva – Andrés Lettieri

Versión: 1.0

Índice

Documento de Modelo de Dominio y Decisiones de Diseño	1
1. Objetivo del documento	3
2. Modelo de dominio	3
2.1. User	4
2.2. Product	4
2.3. Category	5
2.4. Brand	6
2.5. Customer	6
2.6. Vendor	7
2.7. PaymentMethod	8
3. Operaciones	9
3.1. Purchase	9
3.2. PurchaseDetail	10
4. Order	11
4.1. Numeración de Order	12
4.2. OrderDetail	12
5. Stock	13
5.1. Stock actual	13
5.2. StockMovement	14
6. Relaciones principales	15
7. Reglas generales de integridad	16
8. Bajas y conservación del historial	17
9. Auditoría	17
10. Decisiones de diseño	17
11. Criterio para el diseño del MER	18
12. Alcance de este modelo	19

1. Objetivo del documento

El presente documento tiene como objetivo definir el modelo de dominio inicial del sistema y dejar asentadas las principales decisiones de diseño adoptadas para su implementación.

El documento sirve como referencia previa al diseño del Modelo Entidad-Relación (MER) y al posterior desarrollo de las entidades y relaciones utilizando Entity Framework Core.

Las decisiones aquí detalladas buscan representar las necesidades del negocio manteniendo un alcance acorde a un Trabajo Práctico Final, priorizando un modelo simple, consistente y viable dentro de los plazos disponibles.

2. Modelo de dominio

El sistema estará compuesto por tres grandes grupos de entidades:

Seguridad

- User

Entidades maestras

- Product
- Category
- Brand
- Customer
- Vendor
- PaymentMethod

Entidades operativas

- Purchase
- PurchaseDetail
- Order
- OrderDetail
- StockMovement

2.1. User

Representa a los usuarios que utilizan el sistema.

User

id

name

email

password

role

active

createdAt

createdBy

El sistema tendrá inicialmente dos roles:

- ADMIN
- SELLER

El rol será representado mediante un enum y no mediante una entidad independiente.

El usuario administrador tendrá acceso a todos los módulos. El vendedor tendrá acceso principalmente a la gestión de su cartera de clientes y al registro de ventas.

El usuario inicial será creado mediante un script de base de datos.

El campo createdBy se mantendrá como dato de auditoría, pero no se establecerá una relación FK recursiva sobre User.

2.2. Product

Representa los artículos comercializados por la distribuidora.

Product

id

code

barcode
description
categoryId
brandId
purchasePrice
salePrice
stock
minimumStock
active
createdAt
createdBy

El code será ingresado manualmente por el usuario y deberá ser único.

El código de barras será opcional y editable.

Cada producto pertenece obligatoriamente a una categoría y a una marca.

El producto almacenará:

- precio de compra actual;
- precio de venta actual;
- stock actual;
- stock mínimo.

El stock actual se almacenará directamente en el registro del artículo y **no será calculado recorriendo el historial de movimientos**.

No se manejarán imágenes ni unidades de medida dentro del MVP.

2.3. Category

Representa las categorías de artículos.

Category

id
name

createdAt

createdBy

modifiedAt

modifiedBy

El nombre será obligatorio y único.

Las categorías no tendrán baja física. En caso de que posteriormente se requiera ocultar una categoría, se podrá contemplar un mecanismo de baja lógica.

2.4. Brand

Representa las marcas de los artículos.

Brand

id

name

createdAt

createdBy

modifiedAt

modifiedBy

El nombre será obligatorio y único.

Al igual que las categorías, no se contempla la eliminación física de una marca.

2.5. Customer

Representa a los clientes de la distribuidora.

Customer

id

name

document

phone
email
address
city
observations
active
createdAt
createdBy
modifiedAt
modifiedBy

El documento será obligatorio y único.

La dirección se manejará como texto libre y no se creará una entidad independiente para representar domicilios.

No se incluirán en el MVP:

- CUIT.
- Condición de IVA.
- Cuenta corriente.
- Límite de crédito.
- Lista de precios.
- Descuentos.
- Vendedor asignado al cliente.

El cliente podrá ser dado de alta desde el módulo de ventas utilizando el mismo formulario utilizado por el ABM de clientes.

2.6. Vendor

Representa a los proveedores de la distribuidora.

Vendor

id

name
phone
email
address
city
observations
active
createdAt
createdBy
modifiedAt
modifiedBy

El modelo será deliberadamente sencillo.

No se incluirán:

- CUIT.
- Condición de IVA.
- Datos fiscales.
- Cuenta corriente.
- Información bancaria.

No existirá una relación directa entre Vendor y Product.

Un mismo producto puede adquirirse a diferentes proveedores. La relación entre ambos quedará determinada por las operaciones de compra.

2.7. PaymentMethod

Representa los medios de pago utilizados en las ventas.

PaymentMethod

id
name
active

createdAt

createdBy

modifiedAt

modifiedBy

Los medios de pago se almacenarán en una tabla y no en un enum, debido a que pueden surgir nuevos medios de pago sin necesidad de modificar y recompilar la aplicación.

Sin embargo, no se desarrollará un ABM para esta entidad dentro del MVP. Los valores iniciales serán cargados mediante datos iniciales.

Cada venta tendrá un único medio de pago.

3. Operaciones

3.1. Purchase

Representa una operación de ingreso de mercadería.

Purchase

id

number

vendorId

date

total

observations

cancelled

createdAt

createdBy

modifiedAt

modifiedBy

Una compra pertenece a un único proveedor.

La operación deberá contener al menos un PurchaseDetail para poder ser confirmada.

La compra no podrá modificarse una vez confirmada.

En caso de error, deberá ser anulada. La anulación producirá los ajustes correspondientes sobre el stock.

No se utilizará un campo status, ya que no existen estados intermedios de compra dentro del alcance del MVP.

El número de compra será independiente del id técnico de la entidad.

3.2. PurchaseDetail

Representa cada artículo incluido en una compra.

PurchaseDetail

id

purchaseId

productId

quantity

purchasePrice

subtotal

Un mismo producto no podrá repetirse dentro de una misma compra.

Se establecerá una restricción lógica:

UNIQUE(purchaseId, productId)

El purchasePrice será ingresado por el usuario al momento de registrar la operación.

Este precio:

1. Se almacenará en el detalle como valor histórico.
2. Actualizará el purchasePrice actual del producto.

El subtotal será calculado por el backend:

$\text{quantity} \times \text{purchasePrice}$

El frontend podrá mostrar este cálculo de manera informativa mientras se carga la operación, pero el usuario no podrá modificarlo.

El backend será responsable de realizar el cálculo definitivo y persistirlo.

4. Order

Order representa una **venta confirmada**.

No se manejará una entidad independiente denominada Sale.

Tampoco existirán pedidos pendientes dentro del modelo actual.

Order

id

number

customerId

sellerId

paymentMethodId

date

total

observations

cancelled

createdAt

createdBy

modifiedAt

modifiedBy

Una venta tendrá:

- un cliente;
- un vendedor;
- un medio de pago;
- al menos un detalle.

El administrador podrá acceder al módulo de ventas de acuerdo con las reglas de autorización definidas para el sistema.

Vendedor y usuario creador

Order tendrá dos referencias conceptualmente diferentes hacia User:

- sellerId: identifica al vendedor al que se atribuye la venta.
- createdBy: identifica al usuario que registró la operación.

Esto permite, por ejemplo, que un administrador registre una venta correspondiente a un vendedor.

4.1. Numeración de Order

El number será independiente del id.

El número será un valor entero y se mostrará al usuario con ocho dígitos mediante padding.

Por ejemplo:

42 → 00000042

Los ceros no se almacenarán en la base de datos.

El próximo número se determinará tomando el máximo número existente y sumando uno.

4.2. OrderDetail

Representa cada artículo incluido en una venta.

OrderDetail

id

orderId

productId

quantity

salePrice

purchasePrice

subtotal

Un producto no podrá aparecer más de una vez dentro de una misma venta.

Se establecerá:

UNIQUE(orderId, productId)

Precios históricos

El detalle almacenará tanto:

- salePrice: precio de venta al momento de la operación.
- purchasePrice: precio de compra vigente al momento de la operación.

Esto permite calcular posteriormente la ganancia histórica de la operación sin depender de los precios actuales del producto.

Por ejemplo:

salePrice = 1800

purchasePrice = 1200

quantity = 10

La ganancia de la línea podrá determinarse mediante:

$(\text{salePrice} - \text{purchasePrice}) \times \text{quantity}$

El subtotal será calculado por el backend:

$\text{quantity} \times \text{salePrice}$

El frontend podrá mostrar el cálculo de manera informativa.

5. Stock

5.1. Stock actual

El stock actual será almacenado directamente en:

Product.stock

No se calculará recorriendo todos los movimientos históricos.

Cuando una operación modifique el stock, el backend deberá actualizar el stock del producto y registrar el movimiento correspondiente dentro de una misma transacción.

5.2. StockMovement

Representa el historial de modificaciones del stock.

StockMovement

id

productId

type

quantity

referenceId

createdAt

createdBy

modifiedAt

modifiedBy

El campo quantity siempre almacenará valores positivos.

El efecto sobre el stock será determinado por el tipo de movimiento.

Los tipos serán:

PURCHASE

SALE

PURCHASE_CANCELLATION

SALE_CANCELLATION

Conceptualmente:

Tipo	Efecto
PURCHASE	+ stock
SALE	- stock
PURCHASE_CANCELLATI ON	- stock

SALE_CANCELLATION +
stock

referenceId permitirá identificar la operación que originó el movimiento.

Dependiendo del tipo, podrá corresponder a una Purchase o a una Order.

No se establecerá una FK tradicional para referenceId, debido a que puede referenciar diferentes tipos de operación.

6. Relaciones principales

Las relaciones principales del dominio son:

Category 1 — N Product

Brand 1 — N Product

Vendor 1 — N Purchase

Purchase 1 — N PurchaseDetail

Product 1 — N PurchaseDetail

Customer 1 — N Order

User 1 — N Order

PaymentMethod 1 — N Order

Order 1 — N OrderDetail

Product 1 — N OrderDetail

Product 1 — N StockMovement

Los campos createdBy forman parte de la auditoría de las entidades, pero no se considerarán relaciones de dominio en el modelo conceptual.

7. Reglas generales de integridad

Se establecen las siguientes reglas:

Operaciones

- Purchase y Order deben tener al menos un detalle.
- Una operación confirmada no puede editarse.
- Las operaciones pueden anularse.
- Las anulaciones deben conservar el registro histórico.
- Las anulaciones generan el movimiento inverso de stock correspondiente.

Stock

- No se permite vender una cantidad superior al stock disponible.
- El frontend puede informar al usuario sobre el stock disponible.
- El backend debe validar nuevamente el stock antes de confirmar la venta.
- La actualización del stock y el registro del movimiento deben realizarse de forma transaccional.
- Product.stock representa el stock actual.

Precios

- Existe una única lista de precios.
- Product mantiene el precio de compra y el precio de venta actuales.
- El precio de compra puede modificarse al registrar una compra.
- El precio de venta será modificado por el usuario cuando corresponda.
- El sistema podrá advertir cuando el precio de venta sea inferior al precio de compra.
- Las operaciones conservan los precios utilizados al momento de su registro.

Artículos

- code es único.
- barcode es opcional y editable.
- Category y Brand son obligatorias.
- Un artículo puede desactivarse mediante baja lógica.
- No se contempla ajuste manual de stock dentro del MVP.

8. Bajas y conservación del historial

No se contempla la eliminación física de entidades que puedan estar relacionadas con operaciones históricas.

Las entidades maestras utilizarán baja lógica mediante el atributo active.

Las operaciones (Purchase y Order) utilizarán el atributo cancelled para representar su anulación.

El objetivo es conservar la trazabilidad de las operaciones y evitar que la eliminación de un registro afecte la información histórica.

9. Auditoría

Las entidades utilizarán dos campos de auditoría de creación y 2 de actualización:

createdAt

createdBy

modifiedAt

modifiedBy

La auditoría permitirá identificar cuándo y por quién fue creado y modificado cada registro.

10. Decisiones de diseño

Las siguientes decisiones quedan establecidas como base para el posterior diseño de la base de datos:

1. Se utilizará una **base de datos relacional**.
2. El backend será desarrollado en **C# utilizando .NET y Entity Framework Core**.
3. El acceso a datos se realizará mediante DbContext.

4. Las relaciones en C# utilizarán propiedades FK como vendorId, customerId, productId, etc., junto con sus propiedades de navegación cuando corresponda.
5. Role será un enum.
6. PaymentMethod será una tabla, pero no tendrá ABM dentro del MVP.
7. Order representa una venta confirmada.
8. No existirán pedidos pendientes.
9. No existirá una entidad Sale independiente.
10. No existirá relación directa entre Vendor y Product.
11. El stock actual se almacenará en Product.
12. StockMovement conservará el historial de movimientos.
13. Las modificaciones de stock serán transaccionales.
14. Los precios de las operaciones se conservarán históricamente en sus respectivos detalles.
15. Las bajas serán lógicas y las operaciones podrán ser anuladas.
16. No se emitirán comprobantes fiscales.
17. No se calcularán impuestos.
18. No habrá integración con AFIP.
19. No se gestionarán cuentas corrientes.
20. El comprobante de venta será de uso interno de la distribuidora y podrá utilizarse como control de pedido/entrega.

11. Criterio para el diseño del MER

El Modelo Entidad-Relación se construirá tomando este documento como referencia.

En el MER se definirán específicamente:

- claves primarias;
- claves foráneas;
- cardinalidades;

- restricciones de unicidad;
- campos obligatorios y opcionales;
- índices necesarios;
- relaciones entre las entidades.

Los tamaños concretos de los campos y las decisiones propias de implementación de la base de datos se definirán en la etapa de diseño físico y en el script de creación de la base.

12. Alcance de este modelo

El presente modelo representa el **MVP del Trabajo Práctico Final**.

Durante el desarrollo podrán identificarse funcionalidades adicionales propias de una solución real para una distribuidora. Estas podrán documentarse como posibles extensiones futuras, pero no forman parte del modelo obligatorio mientras no sean requeridas por el tutor o resulten necesarias para cumplir los objetivos del trabajo.

Esto permite mantener el proyecto técnicamente viable sin desconocer las necesidades que podrían surgir en una implementación productiva.